Kubernetes Notes

- BY Pavan Kalyan Penchikalapati

Kubernetes:-
---
kubernetes is a popular open source container orchestration software.
k8s was initially developed by google.
k8s to CNCF(Cloud Native Computing Foundation)
k8s is widely used in google applications.

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The open source project is hosted by the Cloud Native Computing Foundation (CNCF).

self managed k8s clusters:-
==========
=> In a self-managed Kubernetes cluster, users have full control over the deployment, configuration, and maintenance of the cluster. They are responsible for setting up and managing the control plane components (master node) and worker nodes.This approach provides greater flexibility, allowing users to customize the cluster to meet specific requirements.
minique --> we can setup single node k8s cluster
kubeadm --> we can setup multi node k8s cluster
kubespray--> we can set up a multi node k8s cluster.
managed k8s clusters:-
==========
In a managed Kubernetes cluster, the cloud provider abstracts the underlying infrastructure complexity. The user delegates operational tasks like cluster provisioning, maintenance, and updates to the managed service.Managed Kubernetes services handle tasks such as automatic scaling, monitoring, logging, and upgrades, reducing the operational burden on the user.
EKS => Elastic k8s service (AWS)
AKS => Azure k8s service
GKE => google k8s engine
IKE => IBM k8s engine

difference between docker swarm and Kubernetes =>
==========

=>docker swarm:- in this approach , in master node server components like api-server , scheduler like many more these are directly installed on os , if any component is going to down , total docker swarm services are down or in case in docker swarm whenever we are maintaining two master nodes at the time one master is down automatically another master node take the work .

==>Kubernetes:- in this approach , in master node server components like apiserver , these are installed as containers in os , if any container is going down automatically it will recreate the same container within fraction of seconds because container management is very easy.

history of kubernetes:-
==========
=> initially in 2003 - google was developing Omega and borg -- run their applications by using light weight container methods.
=> in 2014 google gave the source code of omega and borg product source code to Cloud native computing foundation , it's the sub-foundation of linux foundations.
=>in 2015 CNCF company released Kubernetes on top of omega - borg products with some modifications.
=> the initial version of kubernetes is v1.0

Kubernetes:-
==========

=> Kubernetes is an orchestration tool.it manages the running containers.
=>Kubernetes architecture:-
==========
1.API Server
2.Scheduler
3.controller
4.ETCD Database
5.kube proxy
6.kubelet
7.Container - Docker
8.Pod

Cluster:-
========
A cluster is the fundamental unit of operation in Kubernetes. It's essentially a group of nodes (machines) working together to run containerized applications.

There are two main types of nodes:-
==========
Master node:-
==========
The brain of the operation, responsible for managing the cluster state, scheduling containers, and making overall decisions. Usually just one master node exists.
Worker nodes:-
==========
The workhorses that host and run the containerized applications. Multiple worker nodes can be present, scaling your cluster's capacity.

kubernetes 2 - 1:10s
Kubernetes:-

==> it is called as k8s
==> docker swarm => it is the alternative tool for k8s , it is basically an orchestration system, meaning orchestration is hidden management of containers.

kubernetes:-
==========
==> kubernetes is a portable extensible open source platform for managing containerized workloads and services.
==> kubernetes also facilitates both declarative configuration and automation.
==>Kubernetes provides:-
==========
1)service discovery and load balancing
2)storage orchestration
3)automated rollouts and rollbacks
4)automatic bin packing
5)self healing
6)secrets and configuration management

k8s :-
=====
=> master plane / control plane :-
==========
->API server:-
==========
=> in **control plane** api-server is a core component,api-server is basically expose kubernetes services.this is the heart of the k8s , it takes all requests from the external world .The API server receives commands as request and provides data about the cluster.

=>The single point of contact for interacting with the cluster. Users and tools submit requests, like deploying applications, through the API server.
-> etcd:-
==========
->A highly available distributed key-value store, holding the cluster's desired state (e.g., pod configurations, services). Control plane components and nodes access etcd for updates and coordination.
->Scheduler:-
==========
->Assigns pods to worker nodes based on various criteria (resource availability, node capabilities). It ensures balanced distribution and efficient resource utilization.
Controller Manager: A collection of controllers, each managing specific aspects of the cluster state. Some examples include:
Replication Controller: Ensures a desired number of pod replicas are running across the cluster.
Deployment Controller: Manages application deployments, updating pods incrementally and rolling back if necessary.
Node Controller: Handles node lifecycle events like addition, removal, and health checks.
Cloud-Controller Manager: Interacts with cloud providers for tasks like creating/deleting volumes, load balancers, and managing node instances.

Kubernetes Architecture Deep Dive:
Building on the city analogy, let's explore the intricate details of Kubernetes architecture:

Control Plane:

API Server: The single point of contact for interacting with the cluster. Users and tools submit requests, like deploying applications, through the API server.
etcd: A highly available distributed key-value store, holding the cluster's desired state (e.g., pod configurations, services). Control plane components and nodes access etcd for updates and coordination.
Scheduler: Assigns pods to worker nodes based on various criteria (resource availability, node capabilities). It ensures balanced distribution and efficient resource utilization.
Controller Manager: A collection of controllers, each managing specific aspects of the cluster state. Some examples include:
Replication Controller: Ensures a desired number of pod replicas are running across the cluster.
Deployment Controller: Manages application deployments, updating pods incrementally and rolling back if necessary.
Node Controller: Handles node lifecycle events like addition, removal, and health checks.
Cloud-Controller Manager: Interacts with cloud providers for tasks like creating/deleting volumes, load balancers, and managing node instances.

Worker Nodes:
==========
Each node runs:
Kubelet: Communicates with the API server, receives pod specifications, and manages the lifecycle of containers within those pods. It pulls container images, creates containers, and monitors their health.
Container Runtime: The engine that actually runs the containers (e.g., Docker, containerd). It provides isolation and resource management for each container.
Kube-proxy: Implements network rules and routes traffic to pods based on pod labels and services. It ensures pods can communicate with each other and external services.

what is POD :-
==========

=> POD is the smallest building block in k8s cluster.

=> In k8s, every container will be created inside a pod.

=> POD always runs inside the worker node.

=> POD means a group of containers running on a node.

=> we can create multiple PODS on a single node.

=> every POD will have a unique IP address.

=> we can create POD in ways

1) Interactive approach

```
$ kubectl run --name <pod-name> image=<image-name> --generate=pod/v1
```

2) Manifest YML

=> declarative way

==========

k8s manifest yml:-

==========

```
---
apiVersion: v1
kind: Pod
metadata:
  name: javawebapplicationpod
  labels:
    app: javawebapp
spec:
  containers:
  - name: javawebappcontainer
    image: mrpavaniics/mavenwebapp
    ports:
    - containerPort: 8080
...
```

==========

commands:-

==========

```
$ kubectl get pods
$ kubectl apply -f filename
$ kubectl describe pod podname
$ kubectl logs podname
```

$ kubectl get pods -o wide
$ kubectl get svc => display existing services
$ kubectl apply -f <svc-file> => create k8s service => we can see service information and node port number assigned by k8s.enable node port number security group of pod running node.
$
note:-
======
=> PODS we can't access outside.

=> by default pods are not accessed outside.that means our application is not accessed outside of the cluster.

=> if you want to access the application outside the cluster , we need to expose PODS for outside access using the kubernetes service concept.

kubernetes service:-
==========

=> kubernetes service is used to access pods outside the cluster.
=> we have three types services
  1) Cluster Ip
  2) Node port
  3) Load Balancer

=>we need to write service manifest yml to expose PODS

```
------
apiVersion: v1
kind: Service
metadata:
  name: javawebapplicationpod
spec:
  type: NodePort
  selector:
    app: javawebapp
  Ports:
```

```
  - port:80
    targetPort: 8080
...
```

While both kube-proxy and container network interface (CNI) plugins are crucial components for Kubernetes networking, they handle separate responsibilities:

Kube-proxy:

* **Function:** Network proxy running on each node, ensuring communication between services and pods.
* **What it does:**
  * Translates service definitions into firewall rules (iptables) on the node.
  * Routes traffic to the correct POD based on the rules.
  * Supports several modes: iptables, IPVS (load balancing), and hybrid.
  * Doesn't assign IP addresses or configure the underlying network itself.

CNI plugins:

* **Function:** Specification for configuring network resources in Kubernetes.
* **What it does:**
  * Assigns IP addresses to pods and services.
  * Establishes connectivity between pods across different nodes.
  * Configures different network types like overlay networks (virtual) or underlay networks (existing physical network).
  * Provides a variety of plugins with different features and functionalities.
  * Doesn't handle routing or managing network rules.

In simpler terms:

* **Think of kube-proxy as the traffic controller:** It directs traffic to the right pod based on the destination (service).
* **Think of CNI plugins as the builders of the roads and bridges:** They create the network infrastructure and assign addresses for communication.

Key differences:

| Feature | Kube-proxy | CNI Plugins |

|---|---|---|
Responsibility	Routing traffic, managing network rules	Assigning IP addresses, configuring network
Location	Each node	Each node
Functionality	Limited to routing and firewall rules	Wide range of network configurations
Flexibility	Fixed	Wide variety of plugins available

Working together:

* CNI plugins provide network connectivity details (IP addresses) to kube-proxy.
* Kube-proxy uses this information to set up iptables rules for routing traffic to the corresponding pods.

I hope this helps! Let me know if you have any further questions.

kubernetes reset:-
==========

when we execute the kubernetes reset command , this will reset all components and files , and will remove the etcd member also.

it will executed in 4 phrases:-
==========

==>pre-flight
==>updated-cluster-status
==>remove-etcd-member
==>cleanup-node.

if we want to skip etcd step , we have option to skip by executing below command

kubeadm reset --skip-phases remove-etcd-member

to delete external etcd database

kubeadm reset etcdctl del "" --prefix

Kubernetes directory location

/etc/Kubernetes

etcd directory location

/var/lib/etcd

if you want to remove any worker node from the cluster

kubectl delete node nodeName

if you want to rejoin the worker node

kubeadm join
kubeadm reset

kubernetes upgrade process:-
==========
==> in the kubernetes cluster , the upgradation process one by one is nothing but rolling upgrades.
==> Kubernetes upgradation means it generally involves upgrading three main components
1.kubelet
2,kubeadm
3.kubectl
==>before you start the upgrade , you need to take the backup
1./etc/Kubernetes /manifest

2./var/lib/etcd

In the master node the components are running as pods .
==> before starting the upgrade we want to put the node in drain mode.
==>after performing upgrade activity again we want to perform uncordon activity.it is used for coming out from drain mode.
==> you can only upgrade the minor version of kubernetes not the major one
==> we can only upgrade the minor version .

==> during the upgrade suppose the node goes down or the upgrade didn't go successfully , then you need to forcibly upgrade it , there is no roll-back.
==> whenever the upgradation is not going properly , at the time we want to do force upgrade .Difficult rollback: If issues arise, reverting to the previous version might be complex or impossible, leaving you in a potentially unstable state.

==> kubernetes itself guarantees that it takes the backup of all the files before it starts the upgrade with date and time.
==>network modules or pods will not be upgraded , you need to manually upgrade to a higher version.

kubernetes token and CA-Certificate :-
==========
==>to know kubeadm join
-> kubeadm token list
==> to create token
-> kubeadm token create
==> create token with specific expiry time
-> kubeadm token create --ttl 10m
==> create token with forever without expiry time and date
-> kubeadm token create -ttl 0

kubeadm join --token e4ssip.ucolb6yss1qew24v 172.31.81.5 :6443
--discovery-token-ca-cert-hash
sha256:518ed88df8a36e2f3b51b8cf400550f61c779adcf38ec0598d1454d13c82b3c1

kubeadm join --token e4ssip.ucolb6yss1qew24v 172.31.81.5:6443
--discovery-token-ca-cert-hash
sha256:518ed88df8a36e2f3b51b8cf400550f61c779adcf38ec0598d1454d13c82b3c1

Q) how can avoid the security related consequences like kubeadm join command execute by someone
a)in the master node we have to disable auto approval

ex:-Turning off auto-approval of node client certificates
==> kubectl delete clusterrolebinding kubeadm:node-autoapprove-bootstrap

kubectl get csr :- (certificate signing controller)
==========
=>after executing the kubectl get csr command we will see approved and pending requests from the nodes.

so we want to check all about worker nodes , who wants to joins to master is valid or not

for this

kubectl describe csr name

After this command we will know all about worker nodes.

after checking the nodes configurations , we want to approve valid nodes for that we want to execute this command

kubectl certificate approve name

alternative way to join worker node in cluster is:-
==========

==>kubeadm join --discovery-file path/to/file.conf (local file)

kubeadm join --discovery-file https://url/file.conf (remote HTTPS URL)

kubernetes objects:-
==========
=>Kubernetes objects are configurations that define the desired state of the cluster's workloads and resources. These objects are defined using YAML or JSON files and are submitted to the Kubernetes API server, which then ensures that the cluster's actual state matches the desired state specified in these objects.

every kubernetes object has a specification and the status.
We specify object spec in the format of .yaml file or json file ,and we get status of the object after application of spec.whenever we need to know specification we need to write a .yaml file.

kubectl ==> apiserver

file format => apiserver

kubernetes provides a lot of resources , kubectl api-resources --list of all the resources.

kubectl api-resources => to know all resources

kubectl apply -f filename => apply the .yaml file

kubectl delete 0f filename => to delete the pod

kubectl get pods => to know running pods

kubectl describe pod pname => to know all details of pod

kubectl get pods -o wide => to know the running node and ip address of pod.

kubernetes API:-
==========

=> the heart of the kubernetes control plane is the api server, this API-SERVER EXPOSES HTTP API or RESTful api that end-user and different parts of your cluster , external components communicate each others

kubernetes stores the serialized states of the objects by writing them to the etcd database.

to express the desired state , we would be creating the specification for kubernetes resources
resources:=
==========
1)pods
2)containers
3)ephemeral containers
4)replication controllers

5)replica sets
6)deployments
7)Statefulsets
8)Daemon sets
9)job
10)horizontal pod autoscaler

API groups and versioning:-
==========
=>api groups make it easier to extend the Kubernetes api.the API group is specified in the REST path and is in the form like
=> there are several api groups are there
1) Core Api group
=> rest path /api/v1 --> v1
2)NamedGroup:->
=>api/$groupName/$version

######kubectl#####
=> kubectl is a command-utility for interacting and performing various operations on Kubernetes
=>kubectl as two ways of managing the cluster
1)Imperative method
=> wec create a object by executing the command or kubectl commands
ex:- kubectl get nodes , kubectl get pods
2) declarative method:-
=> we create an object , by executing the .YAML file by using kubectl apply , delete.

how kubectl communicate with api server:-
==========
=>the API server manages communication in kubernetes(for developers it looks like an API gateway.)
=> to achieve the communication , the API server implements RESTFUL API over http and https to perform CRUD Operations to populate and modify K8S API objects,(PODS,service,deployments).

==> when we enter a command on terminal with kubectl, specifying the desired action and resource (e.g., kubectl get pods),after kubectl interacts with kubeconfig file because this file contains necessary information to connect to the API server like cluster ip address ,

auth details using these details kubectl makes a secure and authenticated connection to the api-server,after kubectl translates the user's command into API request (ex:- get pods becomes a GET request), after the API server validates the request , checks user permissions , and interacts with the ETCD key-value store database to access or modify relevant resources, after the API server sends a response back to kubectl , kubectl processes the response and displays it to the user in human-readable format on the terminal.

REST API :-
==========
=>Representational State Transfer API

POD:-
====
A pod is nothing but , it is smallest deployable unit in kubernetes system

Network:

All containers in a Pod share the same network namespace, including the Pod's IP address, network configuration, and routing tables.
This allows direct communication between containers using localhost.
To communicate with external resources outside the Pod, containers must coordinate port usage to avoid conflicts. Kubernetes employs port mapping to expose specific container ports to the Pod's IP address, making them accessible externally.

In Kubernetes, a pause container is a technique used to keep a Pod running even when the primary container within the Pod has terminated.

Here's how it works:

1. **Pod Termination Handling:** When the main container in a Pod completes its task and terminates, the Pod itself may also terminate by default.

2. **Pause Container:** To prevent this from happening and to keep the Pod running, Kubernetes uses a special container called the "pause container." This container essentially does nothing; it just exists to keep the Pod alive.

3. **Keepalive Mechanism:** When the primary container terminates, the pause container continues to run. Because the Pod still has a running container (the pause container), Kubernetes considers the Pod as running. This allows for faster restarts of the primary containers and ensures that other resources, like network endpoints and storage volumes, associated with the Pod are not immediately released.

4. **Pod Termination**: When a new primary container is scheduled to run in the Pod, Kubernetes replaces the pause container with the new container, and then the old pause container terminates. This way, the Pod continues to function seamlessly through container restarts.

In summary, a pause container in Kubernetes is a small, simple container that serves as a placeholder to keep a Pod alive between the termination of its primary containers and the start of new ones. It's a critical component of the Kubernetes lifecycle management system.

How many containers are there ? we don't explicitly define any containers in our pod spec ?

==>
When a Pod is created in Kubernetes, it defaultly includes one container: that is the pause container. This pause container, although not explicitly defined in the Pod spec and not directly visible through standard kubectl commands, is automatically injected by Kubernetes during Pod creation.

Therefore, even if you don't explicitly define any containers in your Pod spec, the resulting Pod will still have at least one container, that is the pause container. This is because the pause container is essential for providing crucial functionalities within a Pod:

When it comes to binding two pods in Kubernetes, using services with ClusterIP is generally the preferred and recommended approach over using pod IPs for several reasons:

Advantages of Services with ClusterIP:

Abstraction: Services provide an abstraction layer that hides the underlying pod IPs and handles load balancing across available pod replicas. This makes your application logic less dependent on specific pod instances, which improves fault tolerance and flexibility.

Scalability: If you need to scale your application by adding or removing pods, the service automatically adjusts to route traffic to the new pods without requiring changes to your application code or configuration.
Stability: Pod IPs can be dynamic and change when pods are restarted or rescheduled. Using a service ensures consistent and stable endpoint information for other pods to connect to.
Security: Exposing pod IPs directly to other pods can potentially expose security risks if they are accessed from outside the cluster. Services provide a controlled access layer, and you can use network policies to further restrict access to specific pods or namespaces.

Recommendation:

In most cases, sticking with services with ClusterIP offers a more robust, scalable, and secure approach for binding pods in Kubernetes. It promotes best practices and reduces the risk of introducing unnecessary complexity or security vulnerabilities.

Kubernetes Services :-
==========
Let's break down how NodePort ,ClusterIP and Load Balancers Services work in Kubernetes and how they facilitate user access to Pods:
==========

ClusterIP:-
==========

Purpose: Exposes a service within the Kubernetes cluster, making it accessible only to other Pods and components inside the cluster.

Working Process:

Service Creation: When a Service of type ClusterIP is created, Kubernetes assigns it a stable virtual IP address (ClusterIP).
Access: Pods within the cluster can discover and access this service using the ClusterIP and the associated Service port.

Traffic Routing: Kubernetes uses internal load balancing mechanisms to distribute traffic across the Pods backing a ClusterIP Service.

nodeport:-
=========
Purpose: Exposes a Service on a specific static port on each Node in the Kubernetes cluster. This makes the service accessible from outside the cluster via any Node's IP address and the assigned NodePort.
Working Process:

Service Creation: When a Service of type NodePort is created, Kubernetes opens a specific port (the NodePort) on every Node in the cluster. NodePorts are typically chosen from a higher port range (e.g., 30000-32767).
Access: External users can reach Pods backing the Service by accessing any Node's IP address and the NodePort.
Traffic Routing: Traffic arriving at the NodePort is internally routed to the ClusterIP of the Service, and then distributed to the appropriate Pods.
When a user wants to connect to a Pod (NodePort):

Outside the cluster: Users from outside the cluster can connect to the service using a Node's public IP address and the NodePort.
Key Points:

ClusterIP is for internal-only access.
NodePort provides external exposure.
NodePort creates a potential security risk by opening the port on all cluster Nodes.
Consider using a LoadBalancer or Ingress for better control.
Choice depends on your use case: Need external access? Go with NodePort or consider even more robust options like LoadBalancer or Ingress. Need only internal access? ClusterIP is perfect.

Load Balancers:=
==========

=>How LoadBalancers Work:

Service of Type LoadBalancer: When you create a Service of type LoadBalancer in Kubernetes, it triggers the cloud provider (e.g., AWS, Azure, GCP) to provision an actual load balancer.
External IP: The cloud load balancer gets assigned a public IP address.
Sync with Kubernetes: The load balancer automatically syncs with Kubernetes to learn about healthy Pods backing the service.
Traffic Distribution: Incoming traffic to the load balancer's external IP is intelligently routed to the Pods backing the Service.
Advantages of LoadBalancers:

Scalability: Handles large volumes of external traffic efficiently.
High Availability: Distributes traffic across multiple Nodes, reducing downtime.
Advanced Features: Supports health checks, session affinity, SSL termination, and more (features vary depending on the cloud provider).
Flexibility: You're not bound to static NodePorts.

How this Works with Previous Concepts:

Nodeports: LoadBalancer Services often still use NodePorts internally to route traffic from the actual load balancer to the Nodes.
ClusterIP: Each Service still gets an associated ClusterIP for internal routing and reference.
Choosing the Right Method

Internal Access: ClusterIP is sufficient.
External Access (Limited): NodePort can work for simple scenarios.
External Access (Production): LoadBalancers are ideal for high availability, control, and scalability.

What is a Kubernetes Endpoint?

Network Interface for Pods: An Endpoint in Kubernetes is a resource that maintains a list of IP addresses and ports for one or more Pods within your cluster. It acts as a point of entry for traffic that needs to reach specific Pods.
Relationship with Services: Endpoints are intrinsically linked to Kubernetes Services. A Service provides a stable virtual IP address and port. When a Service receives traffic, it uses the associated Endpoint object to determine the actual Pod IP addresses and ports to forward the traffic to.

Namespaces:-
==========

=> Virtual Clusters within a Cluster: Namespaces are a way to divide a single Kubernetes cluster into multiple virtual clusters. This lets you organize and manage your Kubernetes resources more effectively.

According to the video, Namespaces are a way to logically partition a Kubernetes cluster into smaller units. They provide a mechanism to isolate resources and workloads from each other, which can be helpful for several reasons:

Organization: Namespaces can be used to group related resources together, making it easier to manage and track them. For example, you could create a namespace for each development, testing, and production environment.
Security: Namespaces can be used to isolate workloads from each other, which can help to improve security. For example, you could prevent a pod in one namespace from accessing resources in another namespace.
Resource quotas: Namespaces can be used to set resource quotas, which limit the amount of resources that can be used by pods in a namespace. This can be helpful for preventing runaway processes from consuming all of the available resources in the cluster.

You can check if a pod is running in a specific namespace using the -n flag with the kubectl get pods command.
You can delete namespaces using the kubectl delete namespace command.
Namespaces are a powerful tool for managing complex Kubernetes deployments. They help to keep resources organized, prevent conflicts, and improve security.

* Resources in one namespace (like Pods, Services, Deployments) don't clash with identically named resources in another namespace. This is great for organizing different projects, teams, or environments on the same cluster.

Security and Access Controls: Kubernetes lets you apply Role-Based Access Control (RBAC) policies at the namespace level. This lets you fine-tune who has permission to view or modify resources within specific namespaces.

Default Namespaces

A Kubernetes cluster has a few built-in namespaces:

default: Where resources go if you don't specify a namespace.
kube-system: Reserved for the Kubernetes system's own components.
kube-public: This namespace is often readable by everyone, used for resources that sometimes need to be cluster-wide accessible.

****
Namespaces create a logical grouping of resources, not a physical constraint.

Within a namespace, pods are not fixed to a specific node. The Kubernetes scheduler can distribute pods across multiple nodes within the same namespace, depending on various factors.

The scheduler can distribute pods across multiple nodes to enhance high availability. If a node fails, pods running on that node can be rescheduled to healthy nodes, ensuring service continuity.

Whenever we create a pod with a manifest file , there are two ways to mention namespace for the pod .
=> in manifest syntax under the metadata we have a provision to give namespace names.
=> when we applying the manifest yaml file at the time we have a way to specify namespace for pod , kubectl apply -f .yaml -n <ns-name> .

****
=>whenever we create a pod without specifying the namespace kubernetes assigned under the default namespace.
=>to see the default namespace
=> kubectl config get-contexts => this command will show the contexts.defaultly k8s give namespace called default under context.
=>so , i want to add pods under a custom namespace called project-1 .
=> we want to add this namespace first under contexts .
=> so kubectl config set-context <context-name> --namespace=<ns-name> --user=kubernetes-admin --cluster=kubernetes. using this command we will add the custom namespaces under contexts.
=> after we want to set a custom namespace as default one .

=> for that kubectl config use-context <context-name>.
=>kubectl config delete-context <context-name> to delete context.

taint:-
=====
=>taints is a mechanism used to control where pods are scheduled.
=>whenever we want to stop assigning pods for a particular node in a cluster , at that time we can use taint as NoSchedule . The taint process will help in the upgrade process .
=>pods are not created on the master node in Kubernetes due to a taint called "NoSchedule". This taint acts as a label that informs the Kubernetes scheduler to exclude the master node when searching for suitable locations to run pods

kubernetes labels:-
==========
==> when we are going to create a pod that we want to create a particular node at that time we use kubernetes labels.

==> actually we want to create pods under one namespace in particular nodes , in this scenario we want to bind kubernetes labels to namespaces .

for this we have two scenarios

=>Using Labels (Incorrect approach):
1)kubectl label nodes node1 environment=production
2)kubectl label nodes node2 environment=production
above two commands will set the label to two nodes .
3)kubectl create namespace my-namespace
4)kubectl label namespace my-namespace environment=production
above two commands will create a namespace and bind two nodes using kubernetes label to namespace.

=>Using Node Selectors(Recommend method)
=> in this method using node selector we assign kubernetes label in pod spec with namespace in metadata .
apiVersion: v1
kind: Pod
metadata:

```
  name: my-pod
  namespace: my-namespace
spec:
  containers:
  - name: my-container
    image: nginx
  nodeSelector:
    environment: production
```

We have edited this before going to bind labels with namespaces.
in /etc/kubernetes /manifests/kube-apiserver.yaml file

from - --enable-admission-plugins=NodeRestriction to -
--enable-admission-plugins=NodeRestriction,PodNodeSelector

after doing this we have give annotations under metadata as
annotations in kubectl edit namespace project-1

```
  annotations:
    scheduler.alpha.kubernetes.io/node-selector: key=value   of label.
```

Resource Quota and Limit Range:-
==========

==>
Kubernetes:

By default, Kubernetes allows a maximum of 110 pods per node for Standard clusters.
This limit can be configured up to 256 pods per node in Standard clusters.
Autopilot clusters dynamically set the limit based on workload density, ranging between 8 and 256 pods per node.

What is a Resource Quota?

A Resource Quota is a Kubernetes object (ResourceQuota) that sets limits on ALL resources a namespace can use. It helps prevent a single team or application from consuming an unfair share of resources, potentially impacting others.

Resource quota is always set to the Namespace,whenever we want to use resource quota we have to create a namespace.under the namespace only we have to set resource quota.

we can only set the resource quota to the namespace only , not node and cluster.

Limit Ranges in Kubernetes:
==========

Limit ranges in Kubernetes are applicable to all three: pods, containers within those pods, and storage requests in PersistentVolumeClaims (PVCs). They enforce constraints on resource usage to ensure efficient allocation and prevent resource exhaustion within a namespace.

ResourceQuota and LimitRange work together to manage resource usage in Kubernetes, but they address different aspects.
ResourceQuota provides a high-level overview and control of resource consumption across a namespace, preventing resource exhaustion.
LimitRange enforces fine-grained limits on how much a pod or container can request and consume resources, ensuring fair resource distribution and preventing individual pods or containers from unfairly consuming resources.

Replication and Pod Controllers:-
==========
==>kubernetes Replication type or kubernetes pod controller type.
==> for replication we have three ways
===>replica-controllers
====>replica-set
=====>deployments
==>replication-controller:-
==========
=> replication-controller work based on labels , for example we have hundreds of pods under replication-controller , at that replication-controller will work based on label , let

assume in 100 pods 20 pods have same label as team:dev , so we can set this label for replication-controller , at that time replication-controller monitor only these label assigned pods only. The remaining 80 pods are assigned with different labels.
=> so replication-control can monitor only one label at a time.it cannot monitor multiple labels at a time.
=> Replication-controller will support rolling updates of the application inside the pod.
==>Replica Set:-
==========
===>monitor the set-based selector , it will monitor more than one label .
===>replica-set will not support any rolling updates of applications inside the pod.
===>Deployments:-
==========
==> deployments will support rolling updates and it will monitor more than one label .
==> Everyone will use only deployments.

difference between replication controller and deployments
==========
=> in the replication controller there is a downtime of the application , where in deployments there is no downtime.
=> in the case of a replication-controller there is no rolling-back option.

While Replication Controllers were historically used for rolling updates in Kubernetes, they are no longer the recommended method. Since Kubernetes version 1.2, Deployments are the preferred way to perform rolling updates. Deployments offer several advantages over Replication Controllers, including:

* **Simplified management:** Deployments manage the lifecycle of your application across multiple revisions.
* **Declarative updates:** You specify the desired state of your deployment, and Kubernetes handles the rollout process.
* **Rollback capability:** In case of issues with a new version, you can easily rollback to a previous deployment.

Even though Kubernetes versions 1.29.1 and earlier support Replication Controllers, it's recommended to migrate to Deployments for rolling updates. Deployments provide a more robust and feature-rich approach for managing application updates in Kubernetes.

Deployment Strategies:-
==========
=>Rolling Deployment
=>Recreate
=> Canary Deployment / Blue Green Deployment / Red black deployment

Each of these deployment strategies serves different purposes and comes with its own advantages and use cases:

1. **Rolling Deployment:**
- **Purpose**:** Rolling deployments gradually replace instances of the old version with the new one, ensuring that the application remains available throughout the update process.
- **Advantages**:** It minimizes downtime and ensures that there's no sudden disruption to the application's availability.
- **Use Cases**:** Rolling deployments are suitable for most production environments where maintaining high availability is crucial.

2. **Recreate:**
- **Purpose**:** Recreate deployments terminate all existing instances before creating new ones with the updated configuration.
- **Advantages**:** It's a simple approach that can be useful when you want to ensure a clean, fresh start with every update.
- **Use Cases**:** Recreate deployments might be suitable for stateless applications or environments where downtime during updates is acceptable.

3. **Canary Deployment:**
- **Purpose**:** Canary deployments gradually shift traffic from the old version to the new one by routing a small percentage of traffic to the new version initially.
- **Advantages**:** Canary deployments allow you to test the new version in a production-like environment with real traffic before rolling it out to the entire user base.
- **Use Cases**:** They are useful for testing new features, gathering feedback, and ensuring the stability of the new version before full deployment.

4. **Blue Green Deployment:**
- **Purpose**:** Blue-green deployments maintain two identical production environments (blue and green), with only one serving live traffic at a time. Updates involve switching traffic from one environment to the other.

- **Advantages**:** Blue-green deployments offer zero-downtime updates and easy rollback capabilities by simply switching traffic back to the previous environment.
- **Use Cases**:** They are suitable for applications with strict uptime requirements and where rolling back quickly in case of issues is essential.

5. **Red Black Deployment (also known as A/B Deployment):**
- **Purpose**:** Red-black deployments are similar to blue-green deployments but involve keeping both versions of the application running simultaneously and routing only a subset of traffic to the new version.
- **Advantages**:** Red-black deployments offer more control over traffic routing and allow for gradual testing of the new version before full deployment.
- ****Use Cases**:** They are suitable for applications that require careful testing and validation of new features before exposing them to the entire user base.

Choosing the right deployment strategy depends on factors such as the application's criticality, downtime tolerance, testing requirements, and infrastructure capabilities.

Why is kubernetes service needed ?
==========

=>Cluster IP, NodePort, and LoadBalancer are all service types in Kubernetes used to expose your applications running in pods to internal or external traffic. They differ in terms of accessibility and how they achieve it.

****1. Cluster IP:****

* ****Role:**** Internal Service Communication
* ****Responsibility:**** Makes pods accessible within the Kubernetes cluster only.
* ****Real-time scenario:**** Imagine you have a backend service and a frontend service within your cluster. The frontend service needs to communicate with the backend service to retrieve data. Cluster IP allows the frontend service to discover and connect to the backend service using its assigned Cluster IP address. This address is only accessible from within the cluster network, not from the outside world.

****2. NodePort:****

* ****Role:**** Exposing Services to External Clients with Limited Access

* **Responsibility:** Exposes pods on a specific port on each node (worker machine) in the cluster. External clients can access the service through this NodePort on any node's IP address.
* **Real-time scenario:** You have an internal service that needs to be accessed by a specific group of users outside the cluster, like a development team testing a new feature. NodePort allows you to expose the service on a designated port (e.g., 31000) on each node. External users can then access the service by connecting to `<node_IP>:31000`. However, this exposes the service on every node, which might not be ideal for widely accessible services.

3. LoadBalancer:

* **Role:** Exposing Services to Public Internet Traffic
* **Responsibility:** Utilizes your cloud provider's load balancer service to create a public IP address that routes traffic to healthy pods behind the service.
* **Real-time scenario:** You have a public-facing web application running in your Kubernetes cluster. A LoadBalancer service creates a public IP address and leverages the cloud provider's load balancer to distribute incoming traffic across the pods associated with the service. This ensures high availability and scalability for your application.

Relationship between these services:

* **Cluster IP** is the foundation for all service types. It establishes the internal communication channel within the cluster.
* **NodePort** builds upon Cluster IP by adding a way to access the service from outside the cluster on a specific port of each node. It's a good option for limited external access.
* **LoadBalancer** leverages Cluster IP and works with your cloud provider's infrastructure to create a publicly accessible entry point for your service. It's the most widely used option for exposing applications to the public internet.

Choosing the right service type depends on your specific needs:

* Use Cluster IP for internal services that only need to communicate with other services within the cluster.
* Use NodePort for services that need to be accessed by a small group of external users with limited access.
* Use LoadBalancer for public-facing services that need to be accessible to anyone on the internet.

default service:-

The kubernetes service primarily facilitates communication between various components within the Kubernetes control plane, such as pods, system services, and applications running on the master node (typically in the kube-system namespace), and the Kubernetes API server. It serves as a gateway to the Kubernetes API server.
When components within the master node, such as kube-controller-manager, kube-scheduler, kube-proxy, or other system services, need to communicate with each other or with the Kubernetes API server, at that time components will use this service.The kubernetes service is typically associated with the IP address of the master node itself.

In the context of Kubernetes networking, the "proxy" mentioned in the description of ClusterIP refers to a component called kube-proxy. Kube-proxy is a network proxy that runs on each node in the Kubernetes cluster. Its primary role is to manage network connectivity to the services exposed within the cluster.

Here's what kube-proxy does:

1. **Service Proxy:** Kube-proxy sets up rules to forward traffic to the appropriate Pods for a given Service. When a client within the cluster tries to access a Service using its ClusterIP, kube-proxy ensures that the traffic is correctly directed to one of the Pods associated with that Service.

2. **Load Balancing:** If multiple Pods back a Service, kube-proxy performs load balancing by distributing incoming traffic among those Pods. This ensures efficient utilization of resources and maintains high availability for the Service.

3. **Health Checks:** Kube-proxy regularly monitors the health of Pods associated with Services. If a Pod becomes unhealthy or unresponsive, kube-proxy stops routing traffic to it until it becomes healthy again.

4. **Endpoint Discovery:** Kube-proxy keeps track of changes to the set of Pods backing a Service. When Pods are added or removed, kube-proxy updates its routing rules accordingly to ensure that traffic is always directed to available and healthy Pods.

In summary, the proxy role of kube-proxy in the context of ClusterIP is to manage network traffic routing and load balancing for Services within the Kubernetes cluster, ensuring reliable communication between clients and the Pods serving the Services.

An init container in Kubernetes is a special type of container that runs before the main containers in a pod start. Its primary purpose is to perform initialization tasks or prepare the environment for the main application containers.

Init containers are useful for scenarios such as:

1. **Prerequisite Setup**: Performing tasks like downloading data, setting up configurations, or initializing databases before the main application starts.

2. **Dependency Resolution**: Ensuring that all dependencies required by the main application are available and properly configured before the application starts.

3. **Delaying Startup**: Waiting for other services or resources to become available before the main application starts.

4. **One-Time Tasks**: Executing one-time initialization tasks that don't need to be part of the main application's lifecycle.

Init containers run to completion before the main containers start. If an init container fails, Kubernetes will restart it until it succeeds. Only after all init containers complete successfully will the main containers in the pod start. This ensures that the main application containers have a properly initialized environment before they begin their execution.

kube-proxy is providing ip address to the pod

kubernetes multi container:-
==========
In a multi container pod , we have to go inside the container , by default we can go to the first container on the pod , to go to other containers in the pod we have to use the below command.

kubectl exec -it <pod-name> -c <container-name> -- /bin/bash

Ingress: The Gateway to Your Kubernetes Services

In Kubernetes, Ingress acts as a single entry point for external traffic reaching your services running within the cluster. It provides a way to expose these services securely and efficiently to the outside world. Here's a breakdown of what Ingress offers:

Purpose:

- Ingress allows you to define rules for routing external traffic to different services based on various criteria like URLs, paths, hostnames, or headers.
- It simplifies managing external access by providing a central configuration point instead of manually exposing each service on individual nodes or relying on a separate load balancer.

Components:

- **Ingress resource:** A YAML file defining the routing rules you want to apply for external traffic. This resource specifies details like:
    - **Backend Services:** The Kubernetes services behind the Ingress that will receive the routed traffic.
    - **Routing Rules:** How to map incoming requests to specific backend services based on various attributes. (e.g., path-based routing using paths or host-based routing using hostnames)
    - **(Optional) TLS configuration:** If you want to terminate SSL/TLS connections at the Ingress for secure communication.

- **Ingress Controller:** A software component responsible for implementing the routing rules defined in the Ingress resource.

    - It interacts with the Kubernetes API server to fetch Ingress definitions and configures the underlying infrastructure (e.g., cloud provider load balancer, network edge router) to implement the desired routing behavior.
    - Popular Ingress controllers include NGINX Ingress Controller, Ambassador, and Traefik.

Benefits of Using Ingress:

- **Simplified Traffic Management:** Define and manage routing rules for all your external services in a single place.
- **Flexibility:** Supports various routing options (path, host, headers) to cater to different scenarios.
- **Load Balancing:** Ingress controllers often integrate with load balancers for distributing traffic across multiple service replicas.
- **Security:** Can be configured to terminate SSL/TLS connections, encrypting communication between external clients and your services.
- **Declarative Management:** The Ingress resource defines the desired state, and the Ingress controller implements it automatically.

Things to Consider:

- **Ingress controllers are separate deployments.** You'll need to choose and deploy a suitable controller for your environment.

- **Ingress resources are currently considered "frozen" in Kubernetes development.** While functional, some features may not be actively extended. The Gateway API is planned to supersede Ingress in the future and address some limitations.

In summary, Ingress plays a crucial role in exposing your Kubernetes services to the external world in a controlled and manageable way. It streamlines traffic routing, enhances security, and simplifies application deployment.

Daemon set:-
==========
In Kubernetes, a DaemonSet ensures that a copy of a specific Pod runs on each node in the cluster. The number of Pods managed by a DaemonSet is equal to the number of nodes in the cluster.

Each node in the cluster that meets the criteria specified by the DaemonSet will have an associated Pod scheduled and running on it. If new nodes are added to the cluster, the DaemonSet controller automatically schedules Pods on those nodes as well, ensuring that there is one Pod per node.

Similarly, if nodes are removed from the cluster, the DaemonSet controller will terminate the corresponding Pods running on those nodes to maintain the desired state of one Pod per node.

This characteristic makes DaemonSets suitable for deploying system-level daemons or agents that need to run on every node in the cluster, such as monitoring agents, log collectors, or networking components.

We want that one of the monitoring pods must be running on all nodes in the cluster all the time.

Certainly! Let's explore the key differences between **Replica Sets and **Daemon Sets** in Kubernetes:**

1. **Replica Sets:**
- **Purpose**:** Replica Sets ensure that a **specified number of identical Pods** are running within your cluster.
- **Scalability**:** You can scale up or down by adjusting the **`replicas`** field in the manifest file.
- **Use Case**:** Replica Sets are commonly used to maintain a set number of Pods, ensuring your application can handle the load and remain available.
- **Example Scenario**:** Suppose you want to run three instances of your web server application to handle incoming requests.

2. **Daemon Sets:**
- **Purpose**:** Daemon Sets ensure that **exactly one copy of a Pod** runs on **all or specific nodes** within your cluster.
- **Automatic Scaling**:** Daemon Sets automatically spin up a Pod on any new node added to the cluster.
- **Use Cases**:**
  - Running a Pod for **log collection** on all nodes.
  - Deploying a **monitoring Pod** on every node.
  - Providing **specific hardware resources** (e.g., GPUs, flash disks) to a subset of Pods.
- **Example Scenario**: You want to deploy a monitoring agent on every node for system performance metrics.

3. **Summary:**
- Replica Sets focus on maintaining a set number of identical Pods for application availability.
- Daemon Sets ensure that critical services or applications run on **every node**, covering the entire cluster.

Remember, there's more to these concepts than what I've covered here. Feel free to explore the Kubernetes documentation for deeper insights and experiment with them in your own environment! 🚀🔍[1][2][3].

Pods are the basic building blocks of applications in Kubernetes. A pod is a group of one or more containers that are deployed together on a Kubernetes node. A container is a standardized unit of software that packages code and all its dependencies so that it can run reliably on any system.

Kubernetes Volumes:-

=>During Run time the pod is generating some data or logs .
There are two types of volumes: ephemeral and persistent.
Ephemeral volumes:-

=>Ephemeral volumes are temporary and are lost when the pod is terminated.
=>Persistent volumes data is permanent and stored in some file system.

Network Storage:-

=>NFS-> network file system
=>Scsi
=>Aws s3 , EBS
=>Azure
=>GCP
=>Red hat gluster

Role Based Access controllers:-

Roles:-
=>Cluster-role
==>Cluster-admin=> if we attach this role to someone , that person will have root access on the cluster
==> self-provisioner => users with this role can create a namespace and delete the namespace.
=>Project/local role- project-level
==>view - a person with this role can see all the resources inside a namespace.
==>edit - a person with this role can create,delete,modify,patch the resources,but the person can't give someone else permission on the namespace and can't even manage the quota and limits.
==>admin- a person with this role can do anything within that namespace.

There are 2 type of role binding

clusterrole === attached ===user === clusterrole
project/local role === attached ===user == role binding

kubectl get rolebinding/clusterrole

Config maps and secrets :-

=>secrets:-

=>secrets is basically its namespace oriented.
=>Secret stores are used to securely store sensitive information such as passwords, API keys, tokens, TLS certificates, and other credentials. These stores employ various encryption techniques to protect the confidentiality of the stored data. Here are some common use cases for secret stores:
1.Key/Value Pairs:
2.Passwords
3.API Keys and Tokens
4.TLS Certificates:
5.File Encryption
secrets always store the information by encoding into base64.

configmap:- basically it is used to store non-sensitive information.

kubernetes probes :-

=> probe is a health check that can be configured to check the health of the container running in the pod.
=>a probe may return the following results
1.success
2.failure
3.unknown

types of probes:
1.liveness probe :- this probe is used to determine if the particular container is running or not, if a container fails the liveness probe , then controller will try to restart the pod in the same node.it is all based on restart policy --, the default restart policy is always.
restart policies:-

1.always
2.never
3.failure
2.readiness probe:- this probe is used to determine whether a particular container is ready to receive the request or not , if this fails then the kubernetes controller will ensure the pod doesn't receive any request.

Implementation of the probes:-

1.Command probes: Command probes are useful for situations where you need custom logic to determine whether a container is healthy or not.
2.http request probes: in this probe, the controller will send get http request to the given address to either port or hostname to perform the probe on the container.we can set the following fields to configure the http probe
* port number
* host name
* http header
* tcp socket probe

Helm is basically used to manage all the manifest files or Yaml files - APplication very easily ..Helm is the package manager in kubernetes

if we have 3 yaml files , all yaml files are interdependent , you need to deploy explicitly one after another , you can make it as a helm chart and you can apply it , so helm takes care of installing all three yaml files easily.

Helm charts are nothing but packaging format for HELM.
charts are nothing but collections of .yaml files , put into a folder

Release:- A release is an instance of a chart running into the kubernetes cluster .

We need kubernetes for following features:
1. High Availability [HA]:-

=> when we run our applications in docker containers , if the container fails then we need to manually start the container or if the node machine fails where all the containers are running on that node so in that case all the containers should be recreated on another machine . so Kubernetes satisfies both of the above.

2.Auto scaling:-

=>containers don't scale on their own.
=>scaling is two types:-
==> 1.vertical scaling
==> 2. horizontal scaling
kubernetes can support both scalings on containers.
3.zero downtime deployments:-

=>kubernetes can handle the deployments with zero downtime deployments, kubernetes can handle rollout (new version) and also roll back (revert to the previous version) with zero downtime.
4.

kubernetes is actually described as a "production grade container management system".

kubernetes Architecture components:-

1.API server
2.Scheduler
3.Controller
4.ETCD database
5.kube-proxy
6.kubelet
7.container runtime
8.Pod
9.node
10.Cloud Control Manager.

Certainly, here's a concise explanation of each component:

1. **API Server:** Acts as the frontend for the Kubernetes control plane. It exposes the Kubernetes API, which allows users and other components to interact with the cluster, such as creating, updating, and deleting resources like pods, services, deployments, etc.

2. **Scheduler:** Responsible for determining which nodes in the cluster should run specific pods based on resource requirements, policies, and constraints. It ensures optimal resource utilization and workload distribution by making intelligent placement decisions.

3. **Controller Manager:** Manages various controllers that regulate the state of the cluster. These controllers include the Node Controller, Replication Controller, Endpoint Controller, and others. Each controller continuously watches the cluster's state and takes corrective actions to ensure that the desired state is maintained.

4. **etcd Database:** A distributed key-value store that serves as the persistent backend for Kubernetes. It stores the entire state of the cluster, including configuration data, metadata, and current resource status. etcd ensures consistency and high availability of data across the cluster.

5. **kube-proxy:** Maintains network connectivity to Kubernetes services by managing network routing, load balancing, and network address translation (NAT) for pods. It implements the Kubernetes service abstraction, allowing clients to access services regardless of the pods' actual locations.

6. **kubelet:** The primary node agent responsible for managing containers on individual nodes. It interacts with the Kubernetes API server to receive pod specifications (manifests), ensures that containers are running and healthy, and reports node status back to the control plane.

7. **Container Runtime:** The software responsible for running containers. Kubernetes supports various container runtimes, such as Docker, containerd, and CRI-O. The container runtime is responsible for pulling container images, creating and managing container lifecycles, and providing isolation between containers.

8. **Pod**: The smallest deployable unit in Kubernetes, consisting of one or more containers that share the same network namespace, IP address, and storage volumes. Pods represent the basic building blocks of applications deployed on Kubernetes, allowing co-located containers to communicate and share resources.

9. **Cloud Controller Manager:** Integrates Kubernetes with cloud providers' APIs to manage cloud-specific resources and functionalities. It abstracts cloud-specific logic from the core Kubernetes components, enabling Kubernetes clusters to run seamlessly across different cloud environments.

Certainly, here's the additional component:

10. **Node:** A node is a worker machine in Kubernetes, responsible for running containers and providing the runtime environment for pods. Each node must have the necessary services to run pods, including the kubelet, container runtime, and kube-proxy. Nodes communicate with the control plane to receive instructions, report status, and manage pod lifecycle. They execute workloads assigned to them by the scheduler and run multiple pods simultaneously. Nodes can be physical or virtual machines and form the underlying infrastructure of a Kubernetes cluster.
These components work together to provide the functionality required for deploying, scaling, and managing containerized applications in a Kubernetes cluster.

There are two primary approaches to scaling applications in Kubernetes:

Horizontal Scaling:

- Involves adding or removing replicas of a Pod (replica set or deployment).
- Increasing replicas distributes the workload across more containers, handling higher traffic or resource demands.
- Decreasing replicas reduces resource consumption during low traffic periods.
- Horizontal scaling is generally considered the preferred approach for stateless applications due to its ease of implementation and high availability.

Vertical Scaling:

- Involves modifying the resource allocation (CPU, memory) for existing Pods within a deployment.
- You can allocate more resources to a Pod if it requires additional processing power or memory.
- Conversely, you can reduce resource allocation for less demanding workloads to optimize resource utilization.
- Vertical scaling can be beneficial for stateful applications or situations where adding more Pods isn't feasible due to cost or licensing constraints.

Deployment Strategies:
Blue Green Deployments:

- This strategy involves maintaining two identical environments, typically called "Blue" (production) and "Green" (staging).
- A new version of the application is deployed to the Green environment first.
- Once thoroughly tested and validated, traffic is gradually shifted from Blue to Green, effectively replacing the old version with the new one.
- This approach minimizes downtime and risk by ensuring a rollback plan is readily available if issues arise in the new version.

Canary Deployments:

- Involves deploying a new version of the application to a small subset of the production traffic (canary).
- The canary Pod(s) run alongside the existing stable version.
- Application performance and health are closely monitored during the canary rollout.
- If the canary performs well, the new version can be gradually rolled out to a larger percentage of the traffic.
- Canary deployments allow for early detection of potential issues in the new version before it impacts the entire production environment.

Self-Healing Mechanisms:

Auto-healing:

- Refers to the ability of Kubernetes to automatically detect and recover from Pod failures.
- The kubelet (agent on each node) monitors the health of Pods using Liveness and Readiness Probes.
- If a Liveness probe fails, the kubelet restarts the Pod.
- If a Readiness probe fails, the Pod is marked unhealthy, and the Kubernetes service stops directing traffic to it.
- Auto-healing ensures your application remains highly available by automatically restarting or replacing unhealthy Pods.
- Pod Enhancements:

Init Containers:

- Special containers that run before the main application container within a Pod.
- They are typically used for performing one-time setup tasks like initializing databases, downloading configuration files, or preparing the environment for the main application.
- Init containers always run to completion before the main container starts.

- They share the same storage volume with the main container, allowing them to share data if necessary.

Sidecar Containers:

- Containers deployed alongside the main application container in a Pod.
- They provide additional functionalities or services that benefit the main application but aren't part of its core functionality.
- **Common sidecar use cases include:**
- **Logging:** Capturing and processing application logs.
- **Monitoring:** Collecting and exporting application metrics for monitoring purposes.
- **Caching:** Caching frequently accessed data to improve application performance.
- **Proxying:** Routing traffic to the main application container.

Service Mesh:

- Not a specific container type, but rather an architectural layer that facilitates service communication within a distributed microservices architecture.
- It utilizes a dedicated network of proxy servers (sidecar proxies) deployed alongside application containers.
-
- Service mesh provides features like service discovery, load balancing, traffic management, and security for communication between microservices.
- Popular service mesh options include Istio, Linkerd, and Consul.

kubectl apply -f .yaml => to create a pod .
kubectl get nodes => to get nodes
kubectl get nodes -o wide => to know more about nodes
kubectl get pods or po => to get pods
kubectl get pods -o wide => to know more about pods
kubectl delete pod podName => to delete pod in node
kubectl delete pod --all => to delete all pods in nodes.
kubectl delete pod --all -n <ns-name> => to delete all pods on particular ns.

kubectl run podname --image=nginx => to create pod like this syntax.
kubectl get endpoints => You can view available Endpoints
kubectl get namespaces or ns => to see namespaces in cluster.
kubectl get pods -A: This command lists all pods in all namespaces, along with the node they are running on.
kubectl describe node <node_name>: This command provides detailed information about a specific node, including the pods currently running on it.
kubectl describe pod <pod-name> => this command will provide detailed information about pod.
kubectl get pods -n or --namespace <namespaces-name> => to display all pods in particiur namespace.
kubectl get pods -n <ns-name> -o wide => to know depth about pods in ns.
kubectl create namespace <namespace-name> => to create a namespace
kubectl describe namespace <namespace-name> => to know deep about namepspace.
kubectl run podname --image=imagename -n namespace-name => to create a pod under particular namespace
kubectl apply -f .yaml -n <ns-name> => to create a pod with ns-name from cli.
kubectl config get-contexts => this command will show the contexts.
kubectl config set-context <contxt-name> --namespace=<ns-name> --user=kubernetes-admin --cluster=kubernetes. using this commad we will add the custom namespaces under contexts.
kubectl config use-context <contxt-name>. => using this commad we will change the context we want.
kubectl config delete-context <context-name> => using this command we will delete contexts.
kubectl exec -it <pod-name> -- /bin/bash > using this command we will go inside the container. this command for a single container in a pod.
kubectl exec -it <pod-name> -c <container-name> -- /bin/bash > using this command we will go inside the particular container we mentioned.
kubectl run -i --tty --image=nginx db-server -- /bin/bash => This command will create a pod based on the specified image and start a shell session inside it.
kubectl port-forward web-server 8081:80 --address 0.0.0.0 => using this command for port forward.
kubectl taint node master-node node-role.kubernetes.io/control-plane- => using this command schedule pods in master node.untainted
kubectl taint node master-node node-role.kubernetes.io/control-plane:NoSchedule =>using this command we can taint master.

kubectl taint node worker-node key=value:NoSchedule or :NoExecute => using this command to taint worker node.
kubectl taint node worker-node key:NoSchedule- or key:NoExecute- => using this command we can untained worker node.
kubeadm token create --print-join-command => this command will show the join command to master.
kubectl get nodes --show-labels => to show labels of every node
kubectl get quota -n <ns-name> => to display Resource Quotas of a particular namespace.
kubectl describes ns <ns-name> => to get information about ns , and resource quota information also.
kubectl describe quota <quota-name> -n <ns-name > .to display only ResourceQuota details of ns.
kubectl explains pod => it can provide information about the available fields and their descriptions within a Pod object.
kubectl explain pod --recursive => The command provides detailed information about the Pod object schema in Kubernetes, including all its nested elements and fields.

kubectl explains pod.spec.containers.image =>This provides a more detailed description of a specific field within the Pod object. Here, it explains the image field under the containers section of the spec property.

kubectl label pod <pd-name> <team=dev>. => this command will take this pod under team=dev label.
kubectl label pod <pd-name> team- => this command will unlabeled pod from rpcn-conler,

kubectl label pod <pd-name> run- => this command will remove the label of pod that comes from default.

kubectl get rc => this command will show a replication-controller .

kubectl delete rc <rc-name> / kubectl delete -f >rc.yaml> => these commands will remove under rc all pods and rc in cluster

kubectl delete rc --cascade=false <rc-name> => this command will remove only the replication-controller , it will exclude pods under this replication controller.

kubectl apply -f deployment.yaml (update the version in yaml) => rolling-update with yaml file.

kubectl rollout undo deployment.apps/nginx-deployment --to-revision=1 => for rollback to previous version.

kubectl get all --show-labels

kubectl describe deployment <deployment-name>

kubectl apply -f deployment.yaml --record=true

kubectl rollout history <deployment-name>

kubectl rollout pause deployment <deployment-name>

kubectl rollout resume deployment <deployment-name>

kubectl scale deployment nginx-deployment --replicas=3 =>This command scales the nginx-deployment to have 3 replicas. Adjust the deployment name and the number of replicas according to your requirements.

kubectl set image deployment <deployment-name> <container-name>=<new-image> =>The kubectl set image command is used to update the image of containers in a deployment.

kubectl set image deployment nginx-deployment nginx=nginx:latest --record=true =>To update the image of a container in a Kubernetes deployment and record the change in the revision history

kubectl edit deployment <deployment-name> =>This command allows you to edit the YAML representation of a Kubernetes resource in your default text editSor and change the version number , automatically rolling update will happen.

kubectl exec -it <pod-name> --/bin/bash => this command is used to go inside the pod.
kubectl expose deployment db-server --type=ClusterIP --port=80 -=-
hostname -i => to know the ip address of the pod.
apt install bind-tools
host google.com

kubectl expose deployment db-server --type=NodePort --port=80
kubectl expose deployment db-server --type=LoadBalancer --port=80

sudo ps -ef | grep -E 'kube|crio|containerd'=>This command should help you identify relevant processes related to Kubernetes and cri-o on your system.

kubectl create secret generic mysqlsecret --from-literal=password=redhat => secret key creation , we have the option to give multiple keys at a time.
kubectl edit secret mysql

kubectl delete secret <name>

kubectl create configmap <name> --from-literal=password=redhat.